HayBale

A Distributed Food Delivery Platform

Group Project Report

Team: 4 Horsemen

Name	Student Number
Andrei Krivoruchko	22328383
Olaf Madynski	22371636
Maxim Koloskov	22348633
Sean Keane	22357956

GitLab Repository:
`https://gitlab.com/ucd-cs-rem/comp30220-2026/4-horsemen`

University College Dublin
Academic Year 2025–2026

1 Final Architecture & Description of Services

1.1 Overview

HayBale is a distributed food delivery platform built using a microservices architecture. The system is made up of four independently deployable services. These services communicate using two different approaches: synchronous REST over HTTP and asynchronous messaging using JMS (ActiveMQ Artemis). All services are containerised with Docker and managed using Docker Compose.

1.2 Services

Service	Port	Owner	Responsibility
Frontend (nginx)	3000	Andrei	Static HTML/JS UI; single entry point for end users
Order Service	8080	Andrei	Place, retrieve, and update food orders
User Service	8081	Maxim	User registration and retrieval
Restaurant Service	8082	Olaf	Restaurant and menu management
Notification Service	8083	Sean	Asynchronous order confirmation via JMS queue
Eureka Server	8761	Sean	Service registry and discovery
ActiveMQ Artemis	8161	Sean	JMS message broker

Table 1: System services, ports, and responsibilities

1.3 Communication Patterns

Synchronous REST (OpenFeign): The Order Service communicates with both the User Service and the Restaurant Service using synchronous HTTP calls. When a new order is placed, the Order Service first checks that the user exists and that the restaurant and menu items are valid before saving the order. Service discovery is handled through Eureka, so there are no hardcoded service URLs in the system.

Asynchronous JMS (ActiveMQ Artemis): After an order is successfully created, the Order Service sends a `Notification` object to the `ORDERS` JMS queue. The Notification Service listens to this queue and processes messages in the background, logging a confirmation when one is received. The two services do not directly depend on each other, as they only interact through the message broker.

HATEOAS: All three REST services use HATEOAS (Hypermedia as the Engine of Application State), which corresponds to Level 3 of the Richardson Maturity Model. Collection endpoints (for example, `GET /orders`) return links to resources rather than

full objects. Individual endpoints (such as `GET /orders/1`) return the full data. `POST` requests return `201 Created` along with a `Location` header pointing to the newly created resource.

1.4 Technology Stack

Technology	Role
Java 21 (Temurin Alpine)	Runtime for all services
Spring Boot 3.4.3	Microservice framework
Spring Cloud Eureka 4.2.0	Service registry and discovery
Spring Cloud OpenFeign 4.2.0	Declarative HTTP client for inter-service calls
Resilience4j 3.2.1	Circuit breaker and fault tolerance
Spring Data JPA + Hibernate 6	Database ORM layer
PostgreSQL 16	Persistent database (Order Service only)
H2 In-Memory	Database for User and Restaurant Services
Apache ActiveMQ Artemis	JMS message broker for async notifications
nginx	Load balancer and static frontend server
Docker + Docker Compose	Containerisation and orchestration

Table 2: Technology stack

Team Member	Contributions
Andrei Krivoruchko	Order Service (REST endpoints, JPA, PostgreSQL, OpenFeign calls, circuit breaker, HATEOAS); Docker Compose configuration; nginx load balancer setup; Eureka client integration; static frontend (HTML/JS)
Olaf Madynski	Restaurant Service (restaurant and menu endpoints, JPA, HATEOAS, Eureka client integration)
Maxim Koloskov	User Service (user registration and retrieval endpoints, JPA, HATEOAS, Eureka client integration)
Sean Keane	Notification Service (JMS consumer, ActiveMQ Artemis integration); Eureka Server setup; ActiveMQ broker configuration; Docker health checks

Table 3: Work division by team member

2 Scalability and Fault Tolerance

2.1 Scalability

The Order Service was designed to support horizontal scaling. Multiple instances can be started using Docker Compose:

```
docker-compose up --scale order-service=2 -d
```

An nginx reverse proxy sits in front of the Order Service instances and distributes requests between them using a round-robin strategy. All instances connect to the same PostgreSQL database, so any order created by one instance is immediately visible to the others. Eureka automatically registers each instance using its container IP, which means service discovery updates without needing manual configuration.

2.2 Fault Tolerance

Circuit Breaker (Resilience4j): The Order Service wraps its OpenFeign calls to the User and Restaurant Services using a circuit breaker. The configuration uses a sliding window of 5 calls and opens the circuit when the failure rate reaches 60%. Once opened, it remains open for 10 seconds before retrying. A `FallbackFactory` is used to distinguish between a service being unavailable (returning `503 Service Unavailable`) and a resource not existing (returning `404 Not Found`). This helps avoid requests hanging and provides a clearer response.

Message Persistence (ActiveMQ Artemis): If the Notification Service is temporarily down, messages are not lost. Instead, they remain in the `ORDERS` queue and are processed once the service becomes available again. This behaviour is handled by Artemis itself and does not require additional implementation.

Docker Health Checks: Health checks are defined for PostgreSQL, the Eureka Server, and the ActiveMQ broker. In Docker Compose, dependent services use `condition: service_healthy`, which ensures they only start once their dependencies are ready. This prevents startup issues where a service tries to connect before another service is fully initialised.

3 Reflection on Experience

This section outlines the main differences between the original proposal and the final implementation, along with the reasons for those changes.

3.1 Authentication Descoped

The original proposal included user authentication as part of the User Service. In the final system, only user registration and retrieval were implemented. Features such as login, JWT tokens, and session handling were not included. This was a conscious decision, as adding full authentication would have introduced significant complexity without adding much value to the overall demonstration. The Order Service still verifies that a user exists before accepting an order, so the core functionality is preserved.

3.2 ActiveMQ Artemis Instead of ActiveMQ Classic

The proposal listed Apache ActiveMQ as the message broker. In the final implementation, ActiveMQ Artemis was used instead, as it is the more modern and recommended version. This required some configuration changes, such as using the Artemis-specific `ActiveMQConnectionFactory` and setting a deserialization whitelist for object messages. However, the overall JMS programming model remained the same.

3.3 nginx Added as Explicit Load Balancer

While the proposal mentioned a load balancer, it did not specify which technology would be used. nginx was chosen as a lightweight and realistic option. It acts as the main entry point on port 8080 and distributes traffic across Order Service instances. One challenge encountered was configuring nginx to use Docker's internal DNS resolver (`127.0.0.11`) so that it could correctly resolve service names as instances were scaled.

3.4 PostgreSQL Only for the Order Service

The proposal suggested using PostgreSQL for all services. In the final version, only the Order Service uses PostgreSQL, while the User and Restaurant Services use H2 in-memory databases. This decision was made to reduce complexity. Since only the Order Service is scaled across multiple instances, it requires a shared persistent database. The other services run as single instances and store simple data, so an in-memory database was sufficient.

3.5 HATEOAS Added

HATEOAS was not included in the original proposal but was added in the final implementation. This brings the system to Level 3 of the Richardson Maturity Model and aligns with the requirements from the module labs. It required some changes to the controllers but improved the overall design of the REST API.

3.6 Docker Health Checks Added

Although Docker and Docker Compose were part of the proposal, health checks were not initially considered. During development, the Order Service sometimes failed to start because PostgreSQL was not ready. Adding health checks and using `condition: service_healthy` resolved this issue by ensuring services only start when their dependencies are available.

3.7 Frontend Added

A simple static frontend was added as an extra feature. It is served by nginx and provides a basic interface for creating users, adding restaurants, and placing orders. This makes it easier to demonstrate the full system without relying on tools like curl, and also shows how HATEOAS links can be followed in practice.